

Object Detection

Viola-Jones Detector

Amirhossein Dadashzadeh

Pui Anantrasirichai

pui-nantheera.github.io/



Object Detection Techniques

- ✓ ☐ **Line and circle detection:** Techniques like the Hough Transform can be used to detect lines and circles in an image, which can indirectly help locate objects with specific geometric shapes.
- ✓ ☐ **Colour-based detection:** In some cases, objects can be detected based on their colour properties. This is especially useful when objects have distinct and consistent colors.
- ✓ ☐ **Template matching:** Using sliding a template over the input image and finding regions where the template best matches the local image content.
- ✓ ☐ **Classifiers with sliding window detectors:** Applying image classification on overlapped patches in the image.
- ✗ ☐ **Deep learning-based object detectors:** Object detector automatically learns image features required for detection tasks, and instance segmentation.

(out of scope in this unit)

Classifiers with sliding window detectors

□ Example Algorithm: **Viola & Jones' Real-time Method**

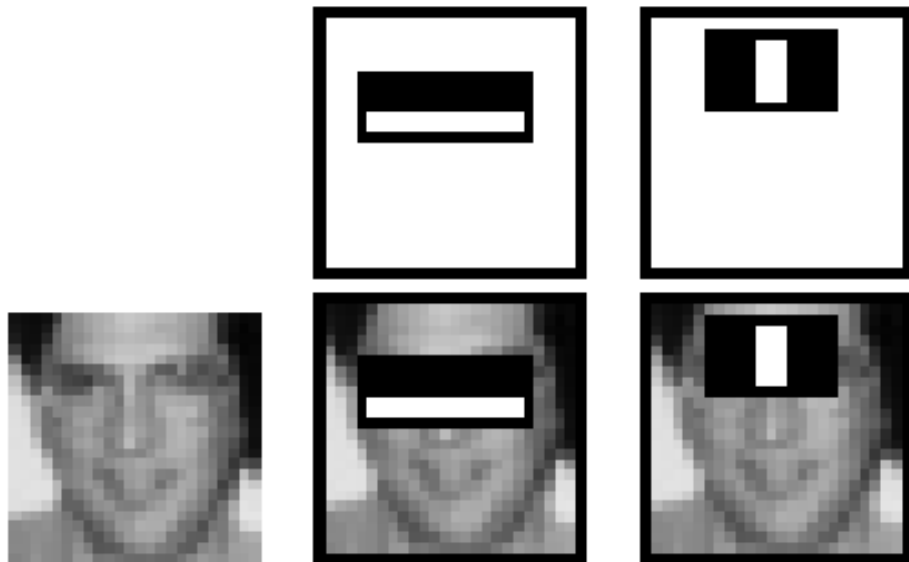
- Sliding Window Detectors
- Haar-like Features
- Feature Extraction and Integral Images
- Weak Classifiers
- Boosting and Classifier Evaluation
- Cascades of Boosted Classifiers



Best description of full details available in consolidated paper by Viola and Jones, International Journal of Computer Vision, 2004

Haar-like Features

Viola & Jones' (2001)



filter 1

-1	-1	1	1
-1	-1	1	1
-1	-1	1	1
-1	-1	1	1

hard edge

feature1 = 2040

0	0	255	255
0	0	255	255
0	0	255	255
0	0	255	255

soft edge

feature1 = 1245

20	120	220	230
25	125	205	225
25	105	220	234
24	110	215	250

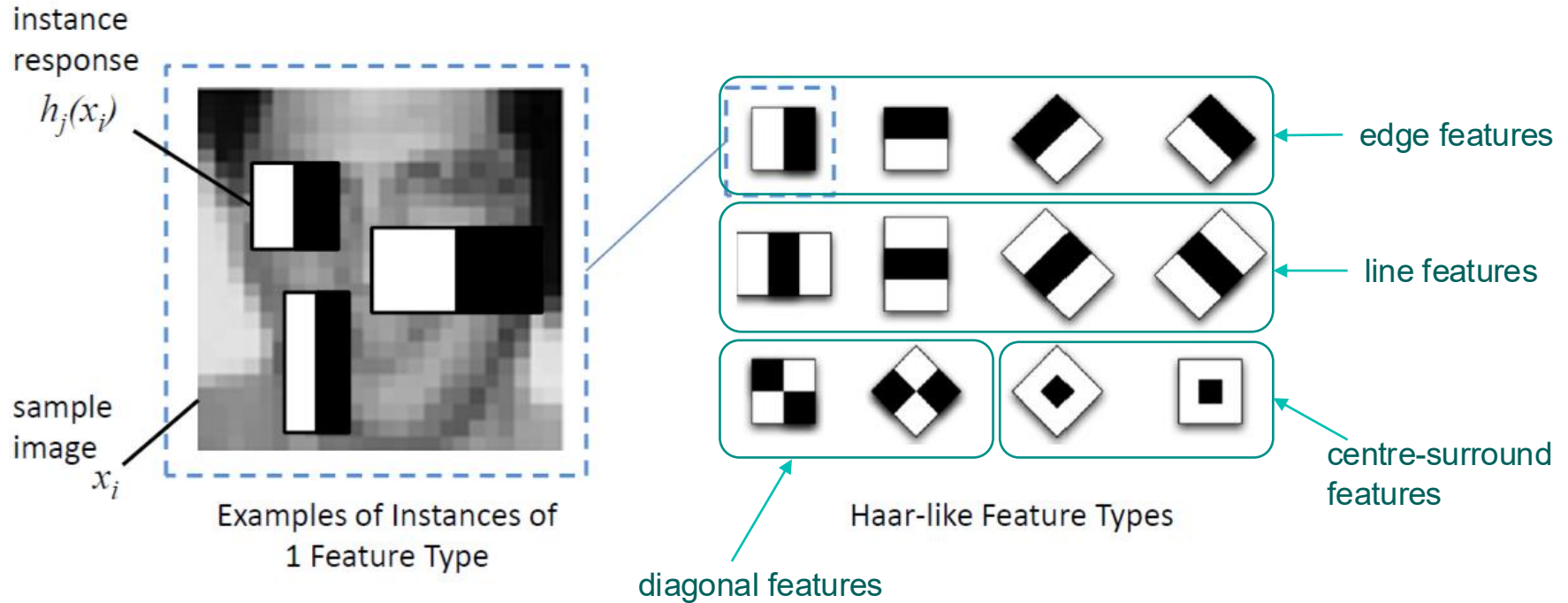
no edge

feature1 = 17

218	230	220	230
200	230	205	225
220	234	220	244
210	250	215	250

feature = sum of white pixels – sum of black pixels

Haar-like Features



Integral Images

I Image

	0	1	2	3	4	5	6	7
0	1	1	1	2	3	1	2	1
1	1	2	0	0	0	3	1	1
2	1	1	1	1	1	2	3	1
3	1	1	1	0	0	0	0	0
4	0	0	0	0	0	0	0	0
5	0	0	0	0	0	0	1	1

II Integral Image

	0	1	2	3	4	5	6	7
0	1	2	3	5	8	9	11	12
1	2	5	6	8	11	15	18	20
2	3	7	9	12	16	22	28	31
3	4	9	12	15	19	25	31	34
4	4	9	12	15	19	25	31	34
5	4	9	12	15	19	25	32	36

Σ

(IMAGE INTEGRATION)

$$\mathbf{II}(-1, y) = 0; \quad \mathbf{II}(x, y) = \mathbf{II}(x - 1, y) + A(x, y);$$

$$A(x, -1) = 0; \quad A(x, y) = A(x, y - 1) + \mathbf{I}(x, y).$$

$$\begin{aligned} \mathbf{x}=0, \mathbf{y}=0: A(0,0) &= A(0,-1) + \mathbf{I}(0,0) = 0 + 1 = 1 \\ \mathbf{II}(0,0) &= \mathbf{II}(-1,0) + A(0,0) = 0 + 1 = 1 \end{aligned}$$

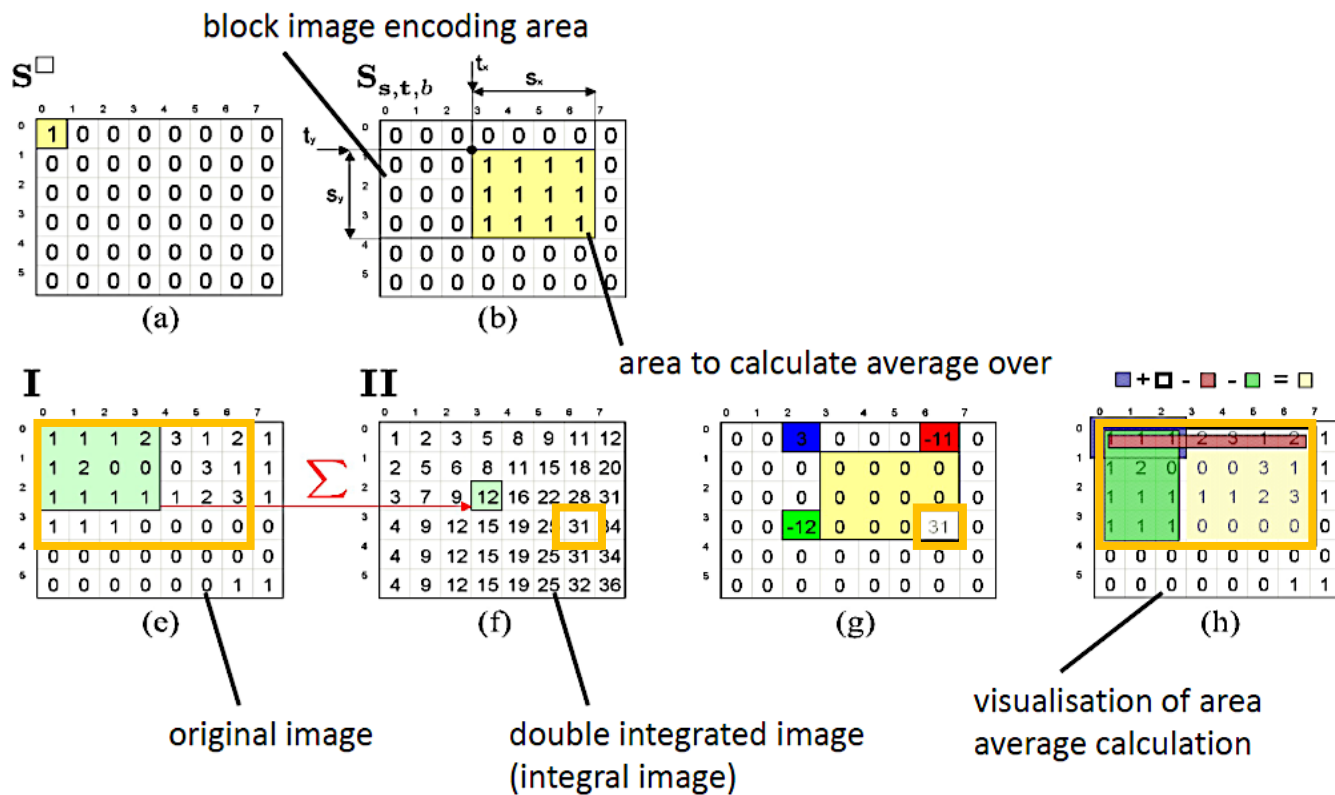
$$\begin{aligned} \mathbf{x}=0, \mathbf{y}=1: A(0,1) &= A(0,0) + \mathbf{I}(0,1) = 1 + 1 = 2 \\ \mathbf{II}(0,1) &= \mathbf{II}(-1,1) + A(0,1) = 0 + 2 = 2 \end{aligned}$$

$$\begin{aligned} \mathbf{x}=1, \mathbf{y}=0: A(1,0) &= A(1,-1) + \mathbf{I}(1,0) = 0 + 1 = 1 \\ \mathbf{II}(1,0) &= \mathbf{II}(0,0) + A(1,0) = 1 + 1 = 2 \end{aligned}$$

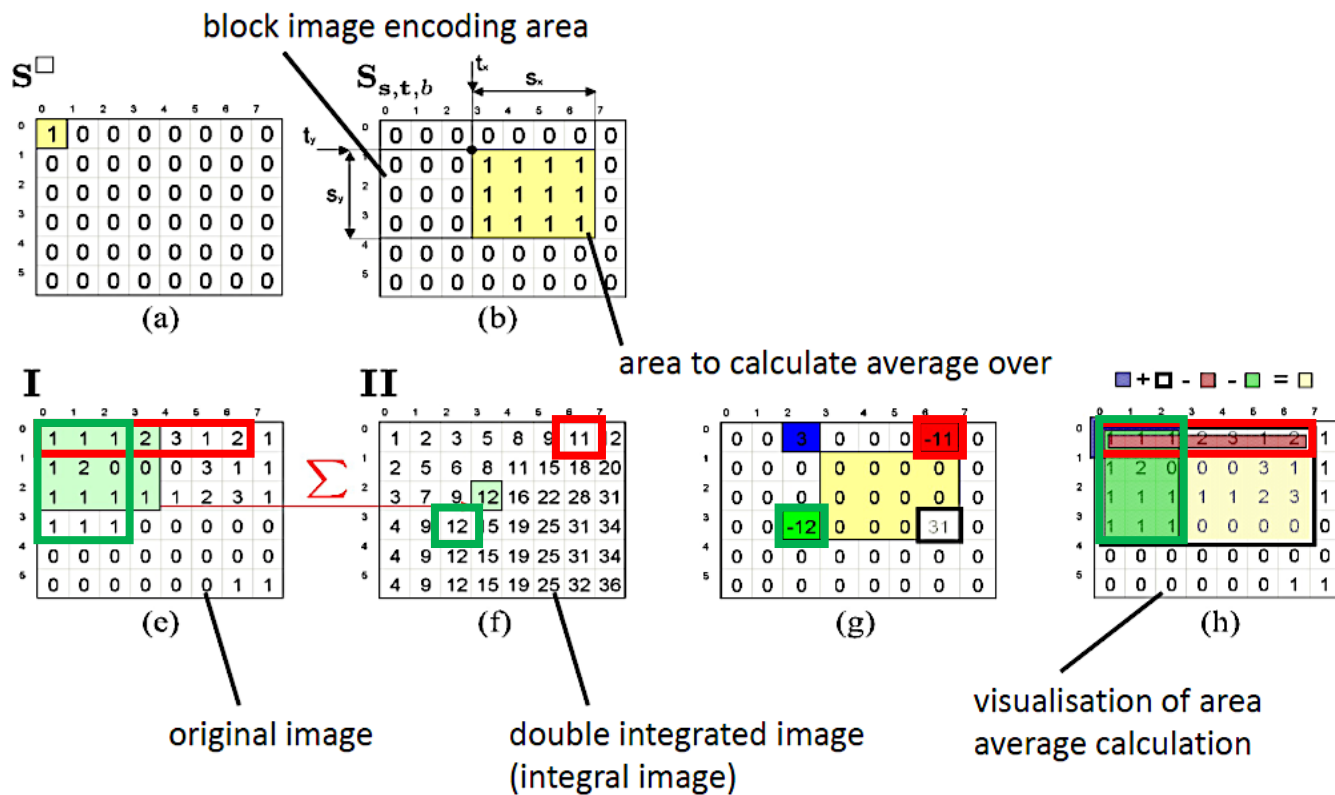
$$\begin{aligned} \mathbf{x}=1, \mathbf{y}=1: A(1,1) &= A(1,0) + \mathbf{I}(1,1) = 1 + 2 = 3 \\ \mathbf{II}(1,1) &= \mathbf{II}(0,1) + A(1,1) = 2 + 3 = 5 \end{aligned}$$

...

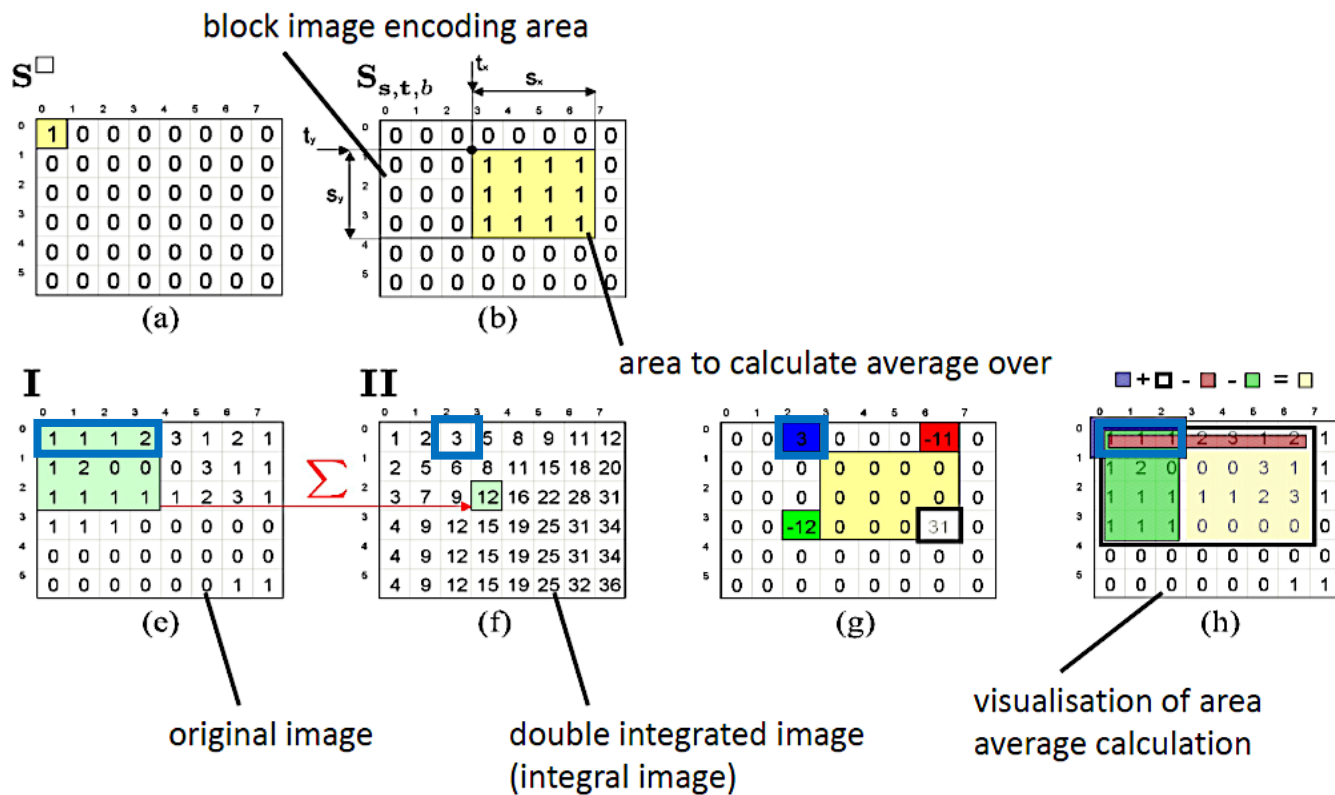
Calculating the Avg Pixel Value of Large Block Fast



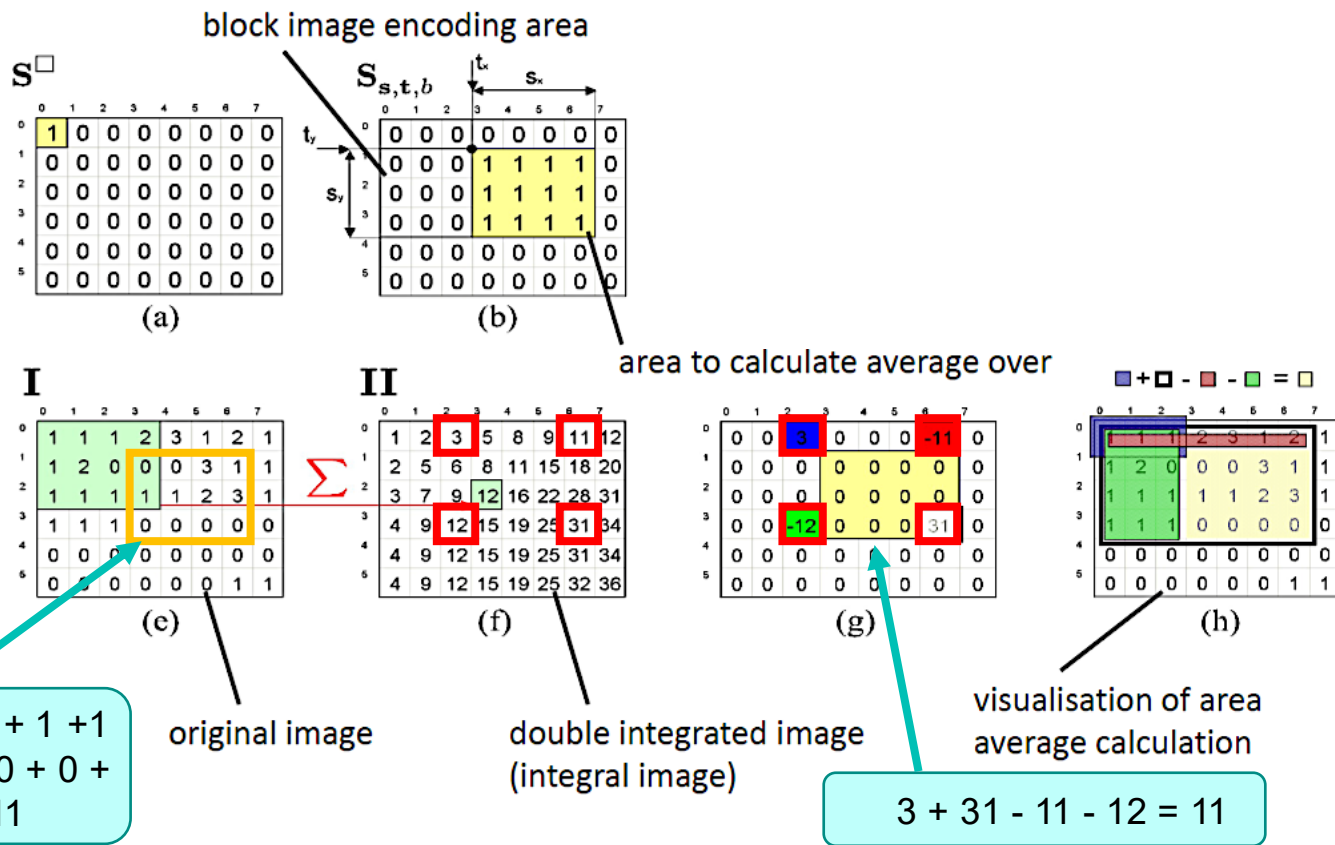
Calculating the Avg Pixel Value of Large Block Fast



Calculating the Avg Pixel Value of Large Block Fast



Calculating the Avg Pixel Value of Large Block Fast



Feature Extraction

x x=2, y=2

	0	1	2	3	4	5	6	7	8	9
0	4	1	1	0	10	0	1	0	0	5
1	3	5	7	1	4	1	8	9	4	0
2	6	4	5	0	1	0	7	8	3	2
3	2	3	4	5	1	1	6	3	4	5
4	9	9	10	1	5	3	1	4	5	4
5	2	2	1	7	4	0	2	1	5	6
6	8	1	1	4	2	3	2	3	1	0
7	0	1	7	0	3	5	6	3	4	1
8	1	5	6	3	5	9	10	4	2	0
9	1	8	3	4	6	3	6	3	3	5

y

Image

-5-0-1-4-5-1-10-1-5-1-7-4-1-4-2-7-0-
 3+0+7+8+1+6+3+3+1+4+0+2+1+3+2+
 3+5+6+3 = **-3**

-1	-1	-1	1	1	1
-1	-1	-1	1	1	1
-1	-1	-1	1	1	1
-1	-1	-1	1	1	1
-1	-1	-1	1	1	1
-1	-1	-1	1	1	1

Haar filter

x

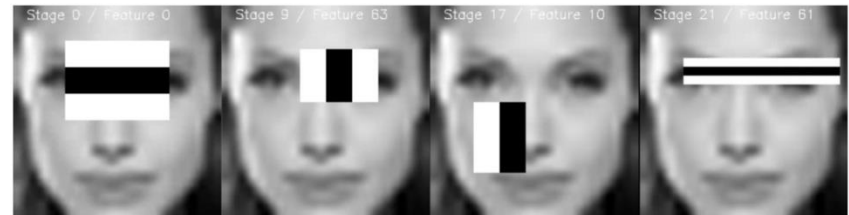
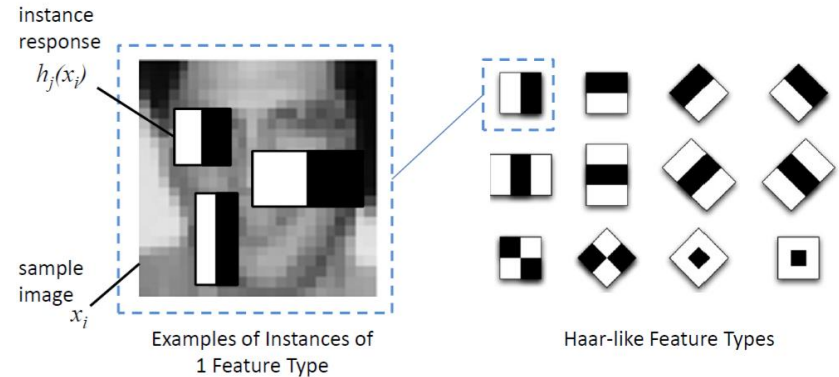
	0	1	2	3	4	5	6	7	8	9
0	4	5	6	6	16	16	17	17	17	22
1	7	13	21	22	36	37	46	55	59	64
2	13	23	36	37	52	53	69	86	93	100
3	15	28	45	51	67	69	91	111	122	134
4	24	46	73	80	101	106	129	153	169	185
5	26	50	78	92	117	122	147	172	193	215
6	34	59	88	106	133	141	168	196	218	240
7	34	60	96	114	144	157	190	221	247	270
8	35	66	108	129	164	186	229	264	292	315
9	36	75	120	145	186	211	260	298	329	357

y

Integral Image

- (144+13-36-60) + (221+36-55-144) = **-3**

Viola & Jones' Real-time Method

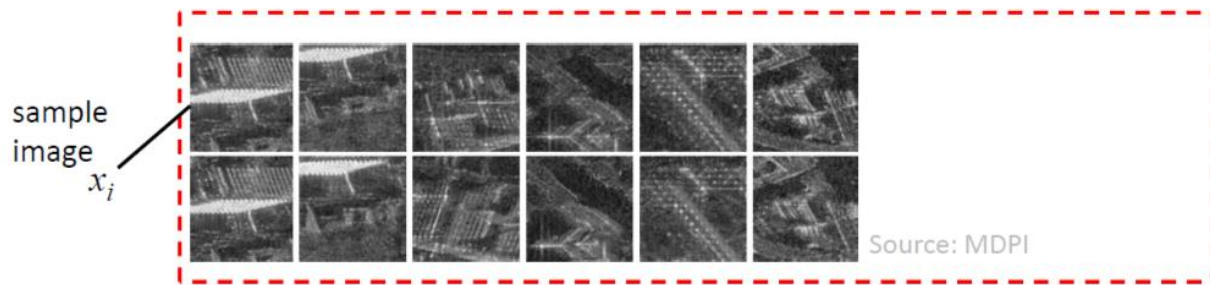


https://medium.com/@Andrew_D/computer-vision-viola-jones-object-detection-d2a609527b7c

Training

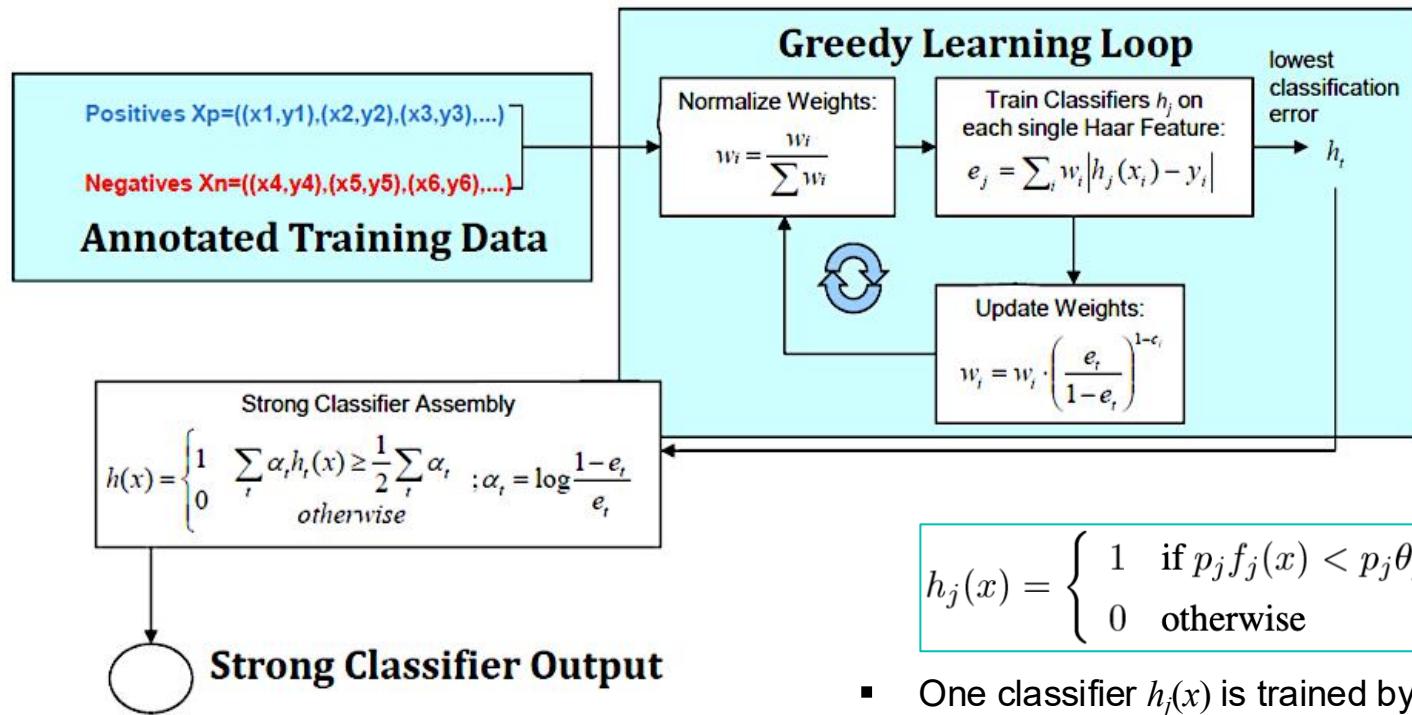


Positive Samples (e.g. FACE) ... $(x_i, y_i = 1), w_i = 1$



Negative Samples (e.g. NO-FACE) ... $(x_i, y_i = 0), w_i = 1$

AdaBoost Classifier

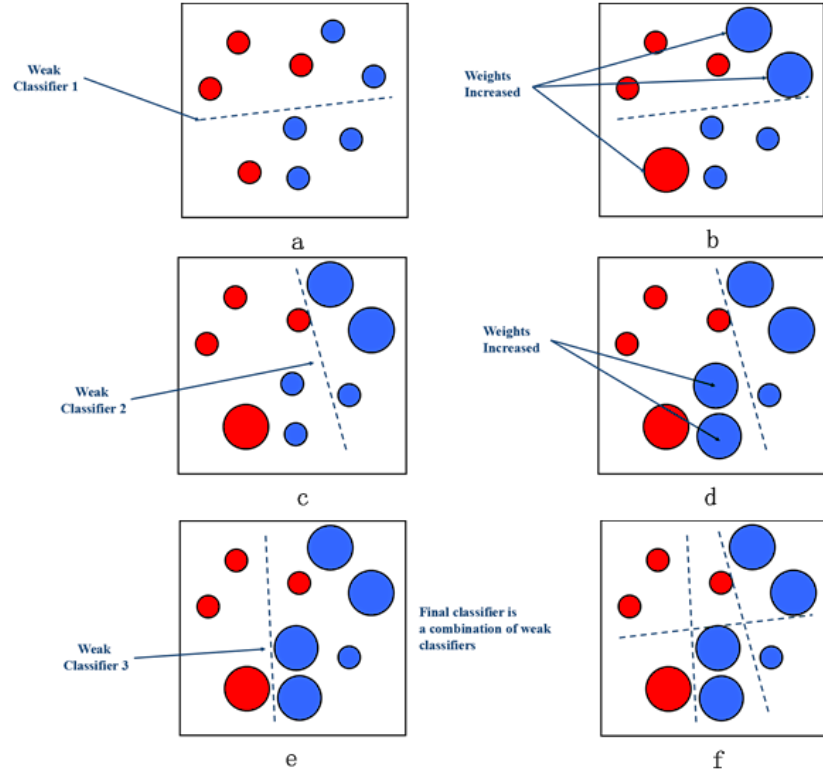
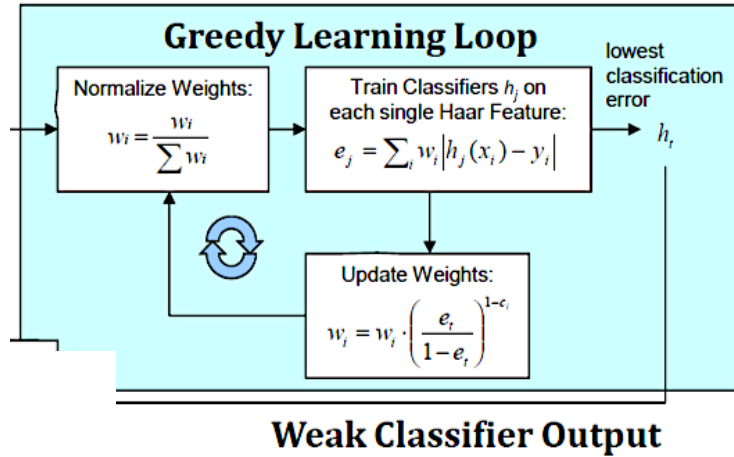


(also see paper by Viola and Jones 2004)

$$h_j(x) = \begin{cases} 1 & \text{if } p_j f_j(x) < p_j \theta_j \\ 0 & \text{otherwise} \end{cases}$$

- One classifier $h_j(x)$ is trained by one feature $f_j(x)$
- θ_j is a threshold
- p_j is a sign + or -

AdaBoost Classifier



Adaboost Algorithm

- Given example images $(x_1, y_1), \dots, (x_n, y_n)$ where $y_i = 0, 1$ for negative and positive examples respectively.
- Initialize weights $w_{1,i} = \frac{1}{2m}, \frac{1}{2l}$ for $y_i = 0, 1$ respectively, where m and l are the number of negatives and positives respectively.

- The final strong classifier is:

$$h(x) = \begin{cases} 1 & \sum_{t=1}^T \alpha_t h_t(x) \geq \frac{1}{2} \sum_{t=1}^T \alpha_t \\ 0 & \text{otherwise} \end{cases}$$

where $\alpha_t = \log \frac{1}{\beta_t}$

- For $t = 1, \dots, T$:

1. Normalize the weights,

$$w_{t,i} \leftarrow \frac{w_{t,i}}{\sum_{j=1}^n w_{t,j}}$$

so that w_t is a probability distribution.

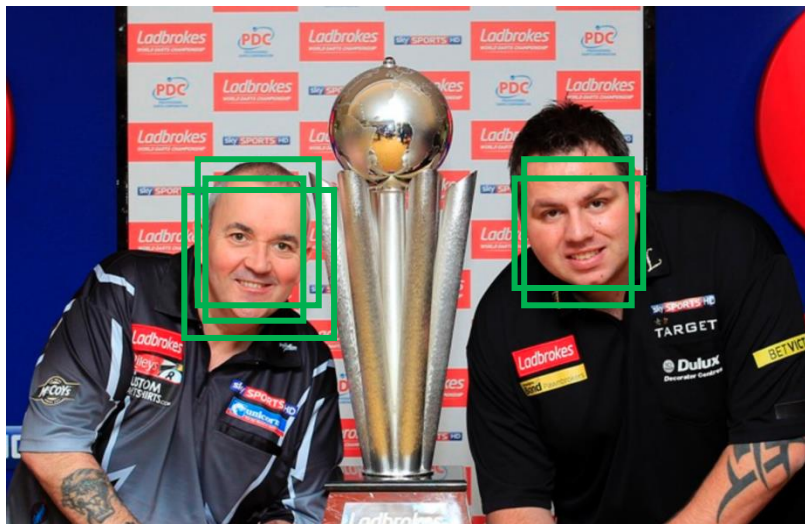
2. For each feature, j , train a classifier h_j which is restricted to using a single feature. The error is evaluated with respect to w_t , $\epsilon_j = \sum_i w_{t,i} |h_j(x_i) - y_i|$.
3. Choose the classifier, h_t , with the lowest error ϵ_t .
4. Update the weights:

$$w_{t+1,i} = w_{t,i} \beta_t^{1-e_i}$$

where $e_i = 0$ if example x_i is classified correctly, $e_i = 1$ otherwise, and $\beta_t = \frac{\epsilon_t}{1-\epsilon_t}$.

Non-Maximum Suppression (NMS)

- A post-processing step to remove redundant detections.



Non-Maximum Suppression (NMS)

- A post-processing step to remove redundant detections.

Algorithm 1 Non-Maximum Suppression Algorithm

Require: Set of predicted bounding boxes B , confidence scores S , IoU threshold τ , confidence threshold T

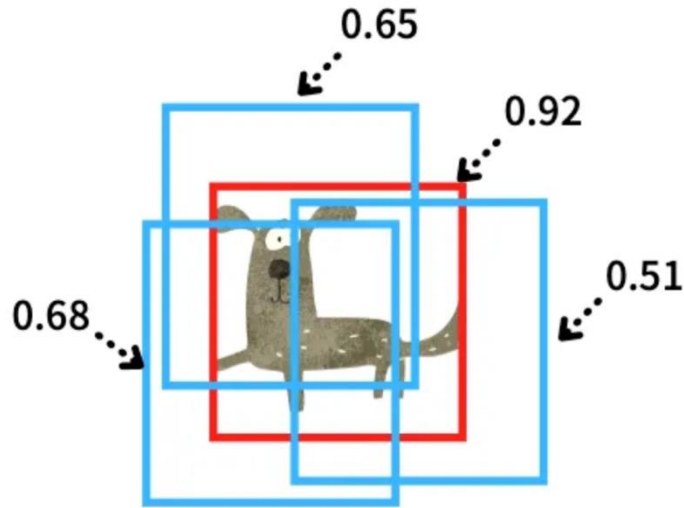
Ensure: Set of filtered bounding boxes F

- 1: $F \leftarrow \emptyset$
 - 2: Filter the boxes: $B \leftarrow \{b \in B \mid S(b) \geq T\}$
 - 3: Sort the boxes B by their confidence scores in descending order
 - 4: **while** $B \neq \emptyset$ **do**
 - 5: Select the box b with the highest confidence score
 - 6: Add b to the set of final boxes F : $F \leftarrow F \cup \{b\}$
 - 7: Remove b from the set of boxes B : $B \leftarrow B - \{b\}$
 - 8: **for all** remaining boxes r in B **do**
 - 9: Calculate the IoU between b and r : $iou \leftarrow IoU(b, r)$
 - 10: **if** $iou \geq \tau$ **then**
 - 11: Remove r from the set of boxes B : $B \leftarrow B - \{r\}$
 - 12: **end if**
 - 13: **end for**
 - 14: **end while**
-

<https://browse.arxiv.org/pdf/2304.00501.pdf>



Non-Maximum Suppression (NMS)

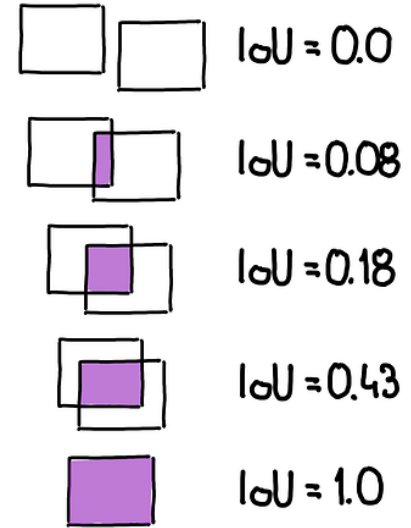


1. take the largest probability box

Intersection over Union

$$\text{IoU} = \frac{\text{INTERSECTION}}{\text{UNION}}$$

The diagram shows two overlapping rectangles. The intersection of the two rectangles is shaded purple and labeled 'INTERSECTION'. The union of the two rectangles is the combined area of both rectangles and is labeled 'UNION'.



2. remove others with IoU score < threshold value.

Non-Maximum Suppression (NMS)

1. Filter Bounding Boxes by Probability:

2. Sort Bounding Boxes by Probability:

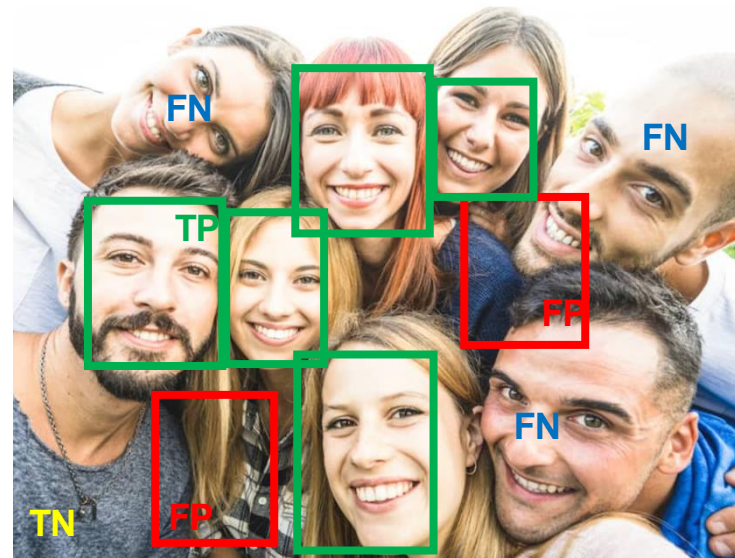
3. Non-Maximum Suppression Loop:



Performance Considerations

		Predicted Class		
		Positive	Negative	
Actual Class	Positive	True Positive (TP)	False Negative (FN) Type II Error	Sensitivity $\frac{TP}{(TP + FN)}$
	Negative	False Positive (FP) Type I Error	True Negative (TN)	Specificity $\frac{TN}{(TN + FP)}$
		Precision $\frac{TP}{(TP + FP)}$	Negative Predictive Value $\frac{TN}{(TN + FN)}$	Accuracy $\frac{TP + TN}{(TP + TN + FP + FN)}$

<https://medium.com/@m.virk1/classification-metrics-65b79bddd776>



<https://www.quickanddirtytips.com/articles/people-or-persons/>

Performance Considerations

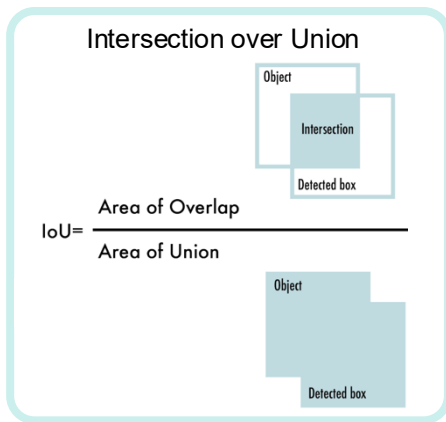
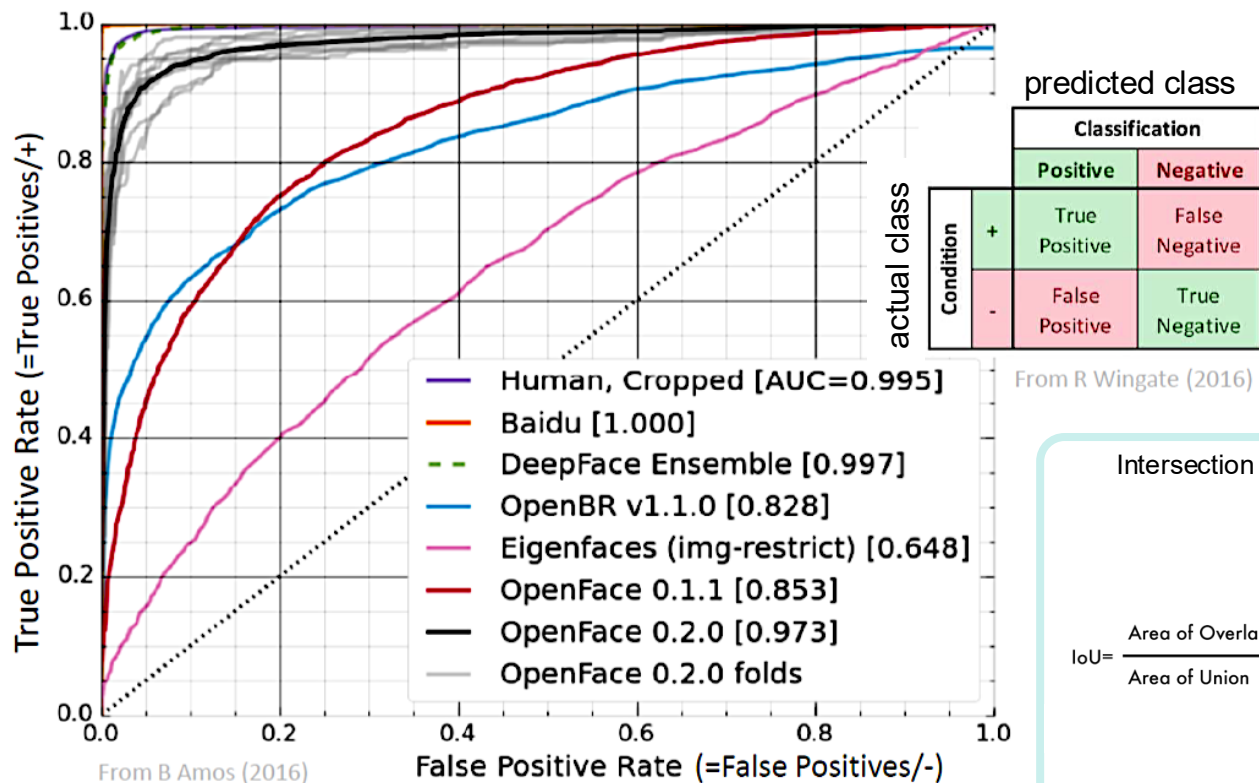
		Predicted Class		
		Positive	Negative	
Actual Class	Positive	True Positive (TP)	False Negative (FN) Type II Error	Sensitivity $\frac{TP}{(TP + FN)}$ recall
	Negative	False Positive (FP) Type I Error	True Negative (TN)	Specificity $\frac{TN}{(TN + FP)}$
		Precision $\frac{TP}{(TP + FP)}$	Negative Predictive Value $\frac{TN}{(TN + FN)}$	Accuracy $\frac{TP + TN}{(TP + TN + FP + FN)}$

F-score

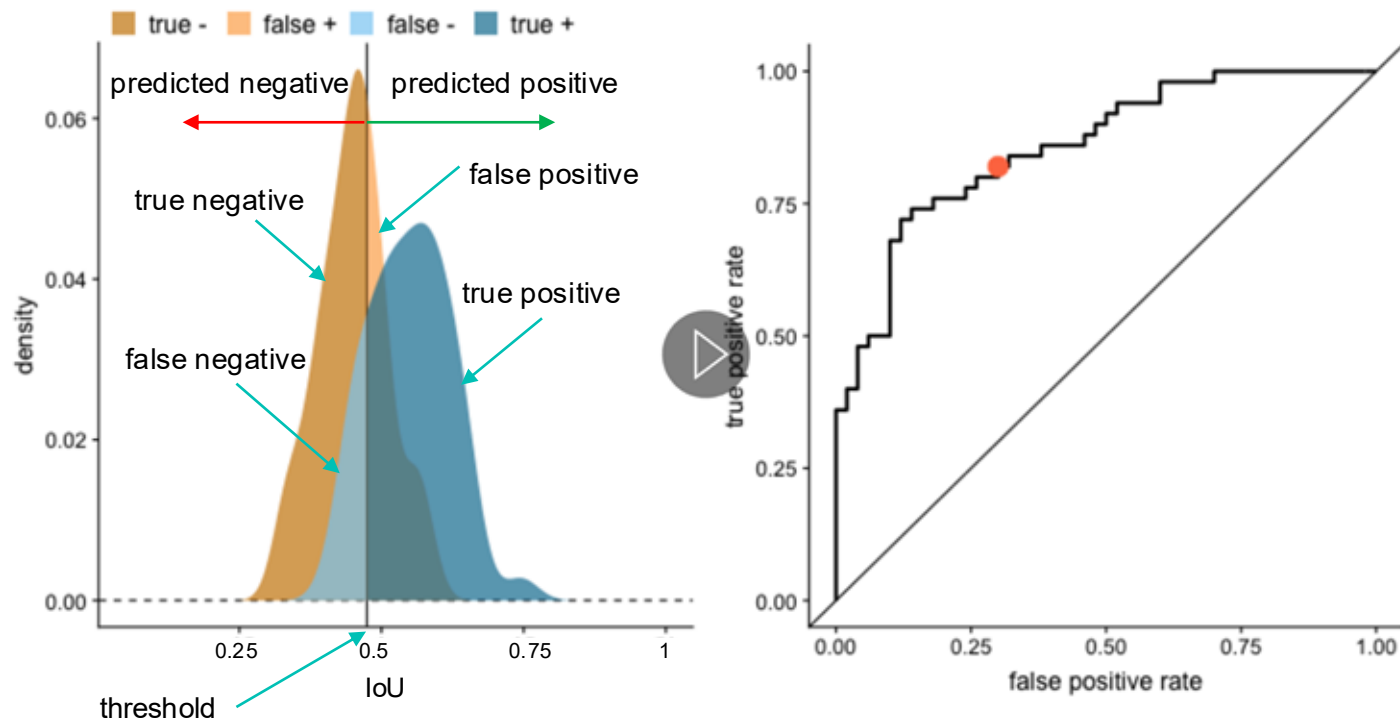
$$F_1 = 2 \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}} = \frac{2tp}{2tp + fp + fn}$$

<https://medium.com/@m.virk1/classification-metrics-65b79bfdd776>

Receiver Operating Characteristic (ROC) Curve

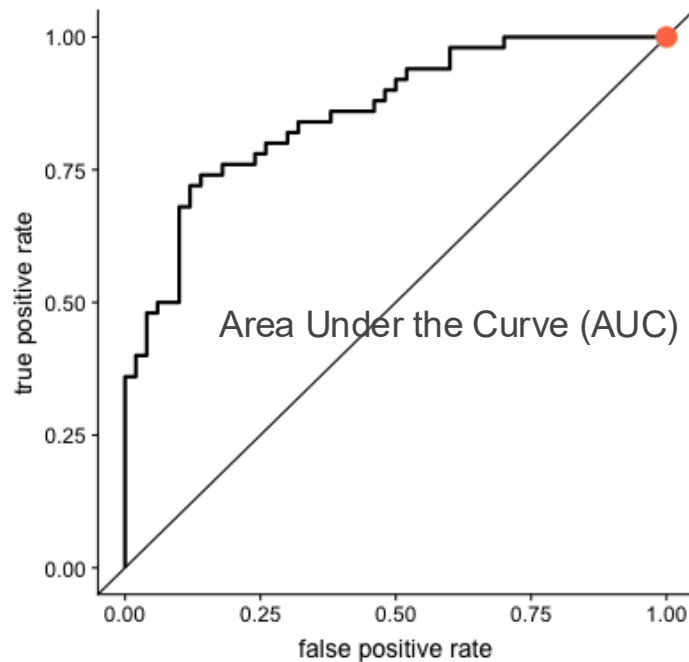
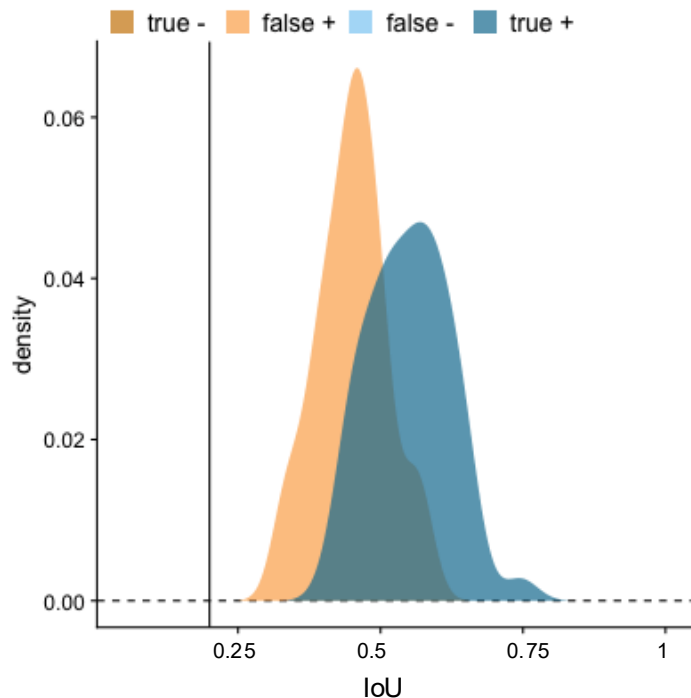


Receiver Operating Characteristic (ROC) Curve



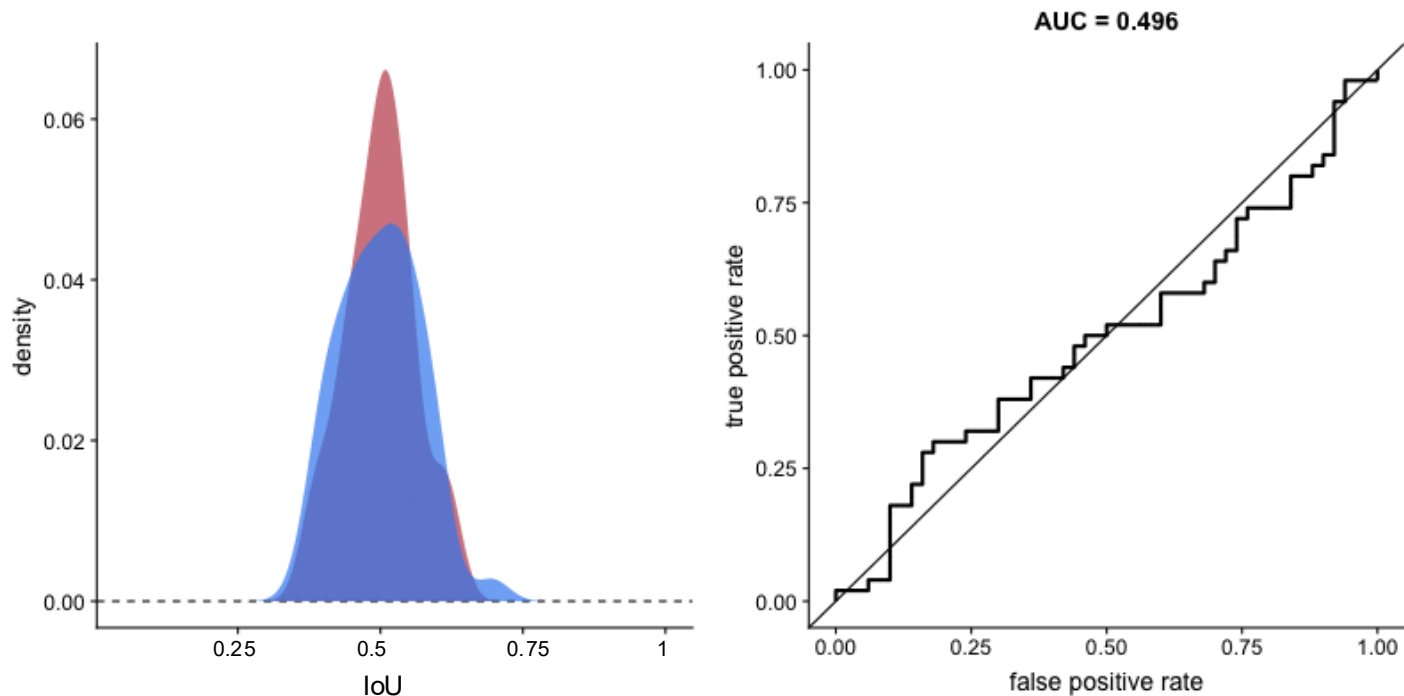
blue: actual positives (e.g. faces)
yellow: actual negatives (e.g. background)

Receiver Operating Characteristic (ROC) Curve



blue: actual positives (e.g. faces)
yellow: actual negatives (e.g. background)

Receiver Operating Characteristic (ROC) Curve



Viola–Jones Today

Still in use

- OpenCV Haar cascades (`cv2.CascadeClassifier`) ship pre-trained face and eye detectors
- Runs in real time on a CPU: good for low-power and embedded devices
- No GPU, no large training set needed at run time

Superseded for accuracy

- CNN detectors (e.g. MTCNN, RetinaFace, YOLO family) cope far better with pose, occlusion and lighting
- Features are learned from data rather than hand-designed Haar boxes
- Deep learning detectors are out of scope for this unit

Ideas that carried over: sliding windows → anchor boxes, cascades → early rejection / coarse-to-fine, plus NMS, IoU and precision–recall evaluation, all still used by modern detectors.

Next

Motion

